

DragonByte Mobile Store

*Sistema Avanzado de Gestión Transaccional, Control de Inventario y Soporte
Técnico en Entornos Oracle PL/SQL*

Asignatura: Bases de Datos Avanzadas

Integrantes: Matheo Franco Zuleta
Brandon Hernández Escobar

Docente Supervisor: Cátedra de Ingeniería de Software

Fecha de Entrega: 5 de Junio de 2026

BOGOTÁ, D.C., COLOMBIA
2026

1. INTRODUCCIÓN

En el entorno contemporáneo de la ingeniería de software, las aplicaciones empresariales demandan arquitecturas de datos capaces de procesar altos volúmenes de transacciones concurrentes garantizando integridad absoluta, mínima latencia y máxima disponibilidad. El presente informe técnico detalla el diseño, desarrollo e implementación del sistema centralizado de gestión para "**DragonByte Mobile Store**", una organización dedicada al comercio y soporte de tecnología móvil. La solución ha sido concebida bajo el paradigma de las **Bases de Datos Avanzadas**, utilizando como motor tecnológico la suite de Oracle (Oracle Live SQL, SQL y PL/SQL).

Tradicionalmente, las reglas operativas de un negocio se delegaban exclusivamente a las capas superiores del software de aplicación (backend), lo cual generaba vulnerabilidades físicas ante fallos de red, redundancias ineficientes y riesgos críticos de consistencia si múltiples clientes accedían al sistema simultáneamente. Este proyecto rompe dicho esquema al aplicar una arquitectura centrada en la base de datos, donde la lógica de negocio, el control de inventarios estricto, los mecanismos de aislamiento concurrente y las auditorías automatizadas residen y se ejecutan directamente en el servidor mediante objetos PL/SQL optimizados. Esto reduce el tráfico de red, erradica errores de mutación y dota a la empresa de una base transaccional sólida con un rendimiento de nivel corporativo.

2. DESCRIPCIÓN DE LA EMPRESA

DragonByte Mobile Store es una organización comercial e industrial de carácter ficticio estructurada para operar en el sector tecnológico competitivo de dispositivos móviles. Su portafolio operativo abarca cuatro pilares estratégicos:

- **Venta de Dispositivos Móviles:** Comercialización de smartphones de diversas gamas y marcas globales.
- **Accesorios Móviles:** Distribución y venta de periféricos, cargadores, fundas de protección y componentes complementarios.
- **Servicio Técnico Especializado:** Diagnóstico preventivo y correctivo a nivel de software y configuraciones avanzadas.
- **Reparaciones de Hardware:** Intervención física de dispositivos (cambios de módulos de pantalla, micro-soldadura, reconstrucción de puertos de carga y sustitución de matrices de componentes defectuosos).

Debido a la diversidad de sus servicios, la empresa interactúa con dinámicas de inventario muy dispares: por un lado, productos estandarizados con stock cuantificable (teléfonos y accesorios); por el otro, órdenes de servicio técnico sujetas a estados variables, asignación de costos estimados y tiempos de entrega críticos. Esto exige un sistema relacional que entrelace armónicamente las ventas directas y el flujo operacional del taller de reparaciones.

2.1. Misión

"Proveer soluciones tecnológicas integrales en movilidad a través de la comercialización de dispositivos de alta calidad y un servicio técnico especializado de vanguardia. Nos comprometemos a optimizar la experiencia de nuestros usuarios garantizando procesos comerciales transparentes, eficientes y seguros, respaldados por una infraestructura de datos robusta que promueva la excelencia operativa y la innovación constante."

2.2. Visión

"Consolidarnos para el año 2031 como la cadena líder en retail tecnológico y soporte técnico especializado de dispositivos móviles a nivel nacional. Nos proyectamos como un referente de transformación digital en la gestión de servicios técnicos, reconocidos por la automatización inteligente de nuestros procesos, la seguridad de la información y la escalabilidad de nuestras plataformas tecnológicas."

3. PROBLEMA IDENTIFICADO

Previo a la conceptualización del sistema propuesto, un análisis del ecosistema operativo básico de DragonByte Mobile Store reveló fallos críticos derivados de la ausencia de una persistencia de datos estructurada y centralizada. Entre los principales problemas analizados desde la perspectiva de la ingeniería de datos se encuentran:

1. **Inconsistencia Crónica de Inventario (Over-selling):** Al procesar transacciones concurrentes sin mecanismos de bloqueo a nivel de fila, se presentaba el fenómeno de venta en negativo. Dos terminales independientes validaban la existencia de una última unidad de un dispositivo de gama alta simultáneamente, confirmando la venta a ambos clientes y generando rupturas de stock físicas y pérdidas reputacionales.
2. **Desacoplamiento Operativo en Servicio Técnico:** Las órdenes de reparación se registraban en medios volátiles o bases de datos no relacionales sin integridad referencial. Esto causaba la pérdida de trazabilidad: dispositivos entregados sin facturar, confusión de propietarios y desconocimiento del historial técnico del hardware.
3. **Vulnerabilidad y Exposición de Datos Sensibles:** El personal de ventas y los técnicos de taller consultaban la misma persistencia de datos plana. Esto exponía información financiera confidencial (costos de adquisición, márgenes de ganancia netos) al personal operativo y comprometía los datos personales de contacto de los clientes.
4. **Falta de Atomicidad en Procesos Comerciales:** Las transacciones complejas (registrar una venta y simultáneamente descontar stock) se ejecutaban en pasos aislados desde el cliente. Si la conexión de red sufría una interrupción a mitad de la operación, la cabecera de la venta se almacenaba pero el detalle del producto o el descuento del inventario no se aplicaba, corrompiendo la base de datos de manera irreversible.

4. VALOR DE NEGOCIO DE LA SOLUCIÓN

La implementación de la arquitectura transaccional en Oracle provee un retorno de inversión estratégico e intangible fundamentado en la estabilidad operativa y la seguridad. El valor corporativo se desglosa en los siguientes ejes estructurales:

EJE ESTRATÉGICO	IMPACTO REAL Y VALOR DE NEGOCIO DE LA SOLUCIÓN SQL/PLSQL
Organización de Datos	Centralización total de las operaciones bajo la Tercera Forma Normal (3FN), erradicando redundancias y anomalías de inserción, actualización o borrado.
Control de Inventario	Automatización estricta por triggers del servidor. Es físicamente imposible vender un accesorio o celular sin disponibilidad en almacén, protegiendo los activos tangibles de la tienda.
Ventas y Reparaciones	Trazabilidad unificada del ciclo de vida del cliente. Cada ingreso al taller técnico genera históricos asociados al identificador único del usuario, facilitando estrategias de fidelización y garantías efectivas.
Seguridad Avanzada	Abstracción total mediante el principio de mínimo privilegio. Los usuarios operativos interactúan con capas lógicas (vistas), blindando el núcleo relacional contra inyecciones de código maliciosas o filtraciones.
Concurrencia ACID	Soporte para múltiples usuarios simultáneos sin riesgo de colisiones de datos. Las transacciones se aíslan mediante políticas explícitas de bloqueo, optimizando los tiempos de respuesta en horas pico.
Reportes y Analítica	Transformación instantánea de datos transaccionales brutos en métricas agregadas de rendimiento gerencial mediante vistas optimizadas para la toma de decisiones financieras oportunas.

5. OBJETIVOS DEL PROYECTO

5.1. Objetivo General

Diseñar e implementar una arquitectura de base de datos avanzada y transaccional para la empresa "DragonByte Mobile Store" utilizando el motor Oracle Live SQL, SQL y el lenguaje procedimental PL/SQL, asegurando el control estricto de inventarios, la gestión de servicios técnicos, el cumplimiento de las propiedades ACID y el blindaje de la información mediante capas lógicas de seguridad.

5.2. Objetivos Específicos

1. Construir un modelo relacional normalizado compuesto por seis entidades estratégicas, haciendo uso estricto de restricciones de llave primaria (PK), llave foránea (FK) y restricciones de verificación (CHECK) para salvaguardar la integridad de los datos.
2. Desarrollar un disparador dinámico (Trigger) a nivel de fila sobre el detalle de ventas que valide el stock real de mercancías en el servidor, impidiendo inserciones transaccionales que vulneren los límites físicos de inventario.
3. Programar procedimientos almacenados (Procedures) modulares y parametrizados con manejo controlado de excepciones para encapsular operaciones complejas como la venta multipartes y el ingreso estructurado de reparaciones técnicas.
4. Implementar técnicas de control de concurrencia avanzadas mediante sentencias de bloqueo exclusivo como **FOR UPDATE NOWAIT** y **WAIT 5**, mitigando cuellos de botella y previniendo interbloqueos (deadlocks) en el motor de base de datos.
5. Diseñar e integrar vistas analíticas y vistas de enmascaramiento que actúen como un filtro de seguridad perimetral para aislar los esquemas de tablas crudas de la manipulación de usuarios no autorizados.

6. REGLAS DE NEGOCIO

Las reglas de negocio representan las políticas operativas de DragonByte Mobile Store traducidas a restricciones lógicas semánticas dentro del servidor de base de datos:

1. **Regla de Dominio de Precios (RN-01):** El precio unitario de cualquier producto o accesorio insertado en el catálogo debe ser estrictamente mayor que cero en cualquier circunstancia (***precio_unitario > 0***).
2. **Regla de Integridad de Stock (RN-02):** El inventario disponible de un artículo comercializable no puede ser negativo bajo ninguna operación. Si la demanda de un cliente supera la existencia actual, la transacción debe ser cancelada en su totalidad de manera automática.
3. **Regla de Atomicidad Comercial (RN-03):** Una transacción de venta se considera válida únicamente si se almacena de forma exitosa su registro de cabecera y la totalidad de sus correspondientes ítems en el detalle de facturación. Cualquier fallo parcial obliga a revertir el proceso al estado inicial.

4. **Regla de Control Operativo de Servicio (RN-04):** Toda orden de reparación técnica debe iniciarse obligatoriamente con un estado por defecto catalogado como 'RECIBIDO' y debe vincularse de manera unívoca a un cliente registrado previamente en la base de datos.
5. **Regla de Privacidad de Costos (RN-05):** La visualización del catálogo de cara a canales públicos o personal de atención básico omitirá por completo los valores internos de coste de importación o márgenes netos, exponiendo únicamente los datos de venta comercial permitidos.

7. ARQUITECTURA DE LA BASE DE DATOS

La arquitectura del sistema está cimentada sobre un modelo relacional fuertemente tipado que cumple con la Tercera Forma Normal (3FN), minimizando la redundancia funcional de datos y garantizando la consistencia ante operaciones de actualización. Las relaciones claves se definen mediante el uso riguroso de restricciones de integridad referencial:

- **Relación Clientes - Ventas (1:N):** Un cliente puede registrar múltiples facturas de compra a lo largo del tiempo, pero cada venta pertenece de forma obligatoria a un único cliente idoneizado por su llave primaria.
- **Relación Ventas - Detalle_Ventas (1:N):** Una venta se compone de uno a N detalles de artículos asociados. La tabla *detalle_ventas* actúa como una entidad de intersección transaccional fuerte.
- **Relación Productos - Detalle_Ventas (1:N):** Un producto del catálogo puede aparecer referenciado en múltiples filas de detalle correspondientes a diferentes facturas comerciales.
- **Relación Clientes - Reparaciones (1:N):** Un cliente puede ingresar múltiples terminales averiados al centro de asistencia técnica, manteniendo la trazabilidad histórica de reparaciones por usuario.
- **Entidad de Auditoría (Log_Cambios):** Tabla desacoplada del flujo relacional directo que almacena los rastros e impactos generados por transacciones críticas, actuando como un libro contable digital del sistema.

8. DICCIONARIO DE DATOS

A continuación se describen las estructuras físicas implementadas para el soporte relacional del sistema DragonByte Mobile Store:

8.1. Tabla: CLIENTES

Almacena la información de identificación y contacto de los clientes de la tienda y del servicio técnico.

COLUMNA	TIPO DE DATOS	RESTRICCIONES	DESCRIPCIÓN
id_cliente	NUMBER	PRIMARY KEY	Identificador único correlativo del cliente.
nombre	VARCHAR2(50)	NOT NULL	Nombres del cliente.
apellido	VARCHAR2(50)	NOT NULL	Apellidos del cliente.
telefono	VARCHAR2(20)	NOT NULL	Número telefónico o de contacto móvil.
email	VARCHAR2(100)	UNIQUE, NOT NULL	Dirección de correo electrónico institucional/personal.
fecha_registro	DATE	DEFAULT SYSDATE	Fecha exacta de alta del cliente en la plataforma.

8.2. Tabla: PRODUCTOS

Contiene el catálogo centralizado de smartphones y accesorios disponibles para comercialización.

COLUMNA	TIPO DE DATOS	RESTRICCIONES	DESCRIPCIÓN
id_producto	NUMBER	PRIMARY KEY	Identificador de catálogo único de la mercancía.
nombre_producto	VARCHAR2(100)	NOT NULL	Nombre comercial del dispositivo o accesorio.
descripcion	VARCHAR2(200)	-	Especificaciones técnicas abreviadas del ítem.
precio_unitario	NUMBER(12,2)	CHECK > 0, NOT NULL	Valor monetario de venta al público general.
stock	NUMBER	CHECK >= 0, NOT NULL	Cantidad física de unidades disponibles en bodega.
categoria	VARCHAR2(30)	NOT NULL	Clasificación (e.g., 'CELULARES', 'ACCESORIOS').

8.3. Tabla: VENTAS (Cabecera)

Registra los metadatos globales de las operaciones comerciales consolidadas.

COLUMNA	TIPO DE DATOS	RESTRICCIONES	DESCRIPCIÓN
id_venta	NUMBER	PRIMARY KEY	Número correlativo único de la factura de venta.
id_cliente	NUMBER	FOREIGN KEY	Vinculación obligatoria con la tabla CLIENTES.
fecha_venta	DATE	DEFAULT SYSDATE	Estampa de tiempo de la transacción comercial.
total_venta	NUMBER(12,2)	DEFAULT 0	Monto económico acumulado total de la operación.

8.4. Tabla: DETALLE_VENTAS

Entidad desagregada que detalla cada artículo incorporado en una venta específica.

COLUMNA	TIPO DE DATOS	RESTRICCIONES	DESCRIPCIÓN
id_detalle	NUMBER	PRIMARY KEY	Identificador correlativo interno de la línea de detalle.
id_venta	NUMBER	FOREIGN KEY	Referencia cruzada a la cabecera de la VENTA.
id_producto	NUMBER	FOREIGN KEY	Referencia cruzada al ítem vendido (PRODUCTOS).
cantidad	NUMBER	CHECK > 0, NOT NULL	Unidades adquiridas por el cliente en esa línea.
precio_unitario_aplicado	NUMBER(12,2)	NOT NULL	Precio real congelado en el momento de la venta.

8.5. Tabla: REPARACIONES

Gestiona las órdenes de soporte técnico, estados de dispositivos y costos asociados del taller.

COLUMNA	TIPO DE DATOS	RESTRICCIONES	DESCRIPCIÓN
id_reparacion	NUMBER	PRIMARY KEY	Número único de ticket de asistencia técnica.
id_cliente	NUMBER	FOREIGN KEY	Propietario del dispositivo (CLIENTES).
dispositivo	VARCHAR2(100)	NOT NULL	Marca y modelo del terminal en reparación (e.g., iPhone 15).
descripcion_fallo	VARCHAR2(300)	NOT NULL	Síntomas o diagnóstico inicial reportado por el cliente.
estado	VARCHAR2(20)	CHECK IN (...)	Estados: 'RECIBIDO', 'EN_PROCESO', 'REPARADO', 'ENTREGADO'.
costo_estimado	NUMBER(12,2)	CHECK >= 0	Valor monetario proyectado del soporte técnico.
fecha_ingreso	DATE	DEFAULT SYSDATE	Ingreso físico del terminal al laboratorio de servicio.

8.6. Tabla: LOG_CAMBIOS (Auditoría)

Registro histórico secuencial de operaciones internas automáticas para seguridad de datos.

COLUMNA	TIPO DE DATOS	RESTRICCIONES	DESCRIPCIÓN
id_log	NUMBER	PRIMARY KEY	Consecutivo único de la auditoría.
tabla_afectada	VARCHAR2(30)	NOT NULL	Nombre de la tabla modificada por el sistema.
operacion	VARCHAR2(10)	NOT NULL	Tipo de manipulación (INSERT, UPDATE, DELETE).
usuario	VARCHAR2(50)	DEFAULT USER	Identidad del usuario de base de datos que ejecutó la acción.
fecha	DATE	DEFAULT SYSDATE	Estampa cronológica exacta del suceso.
descripcion	VARCHAR2(400)	-	Detalle textual explicativo del impacto de la transacción.

9. EXPLICACIÓN DE LOS TRIGGERS (DISPARADORES)

En bases de datos avanzadas, los disparadores actúan como guardianes automatizados de la integridad semántica. En DragonByte Mobile Store se implementó el trigger estructural denominado *trg_validar_stock_detalle*. Este trigger está clasificado técnicamente como un disparador a nivel de fila (**FOR EACH ROW**) configurado para activarse inmediatamente antes de cualquier evento de inserción o modificación en la tabla de quiebre operativo (**BEFORE INSERT OR UPDATE ON detalle_ventas**).

Propósito y Mecanismo Interno: Cuando una aplicación intenta registrar una fila en el detalle de una venta, el trigger intercepta los datos de entrada (*:NEW.id_producto* y *:NEW.cantidad*). Acto seguido, realiza una consulta interna sobre la tabla *productos* para determinar las existencias reales en almacén. Si el inventario es insuficiente (*stock < :NEW.cantidad*), el trigger aborta la ejecución mediante la invocación de la instrucción del sistema *RAISE_APPLICATION_ERROR*, enviando un código de error personalizado y revirtiendo cualquier cambio previo. En caso de contar con stock disponible, el trigger ejecuta de forma autónoma una sentencia de actualización para descontar las unidades vendidas directamente del inventario, blindando el sistema contra desfases mecánicos humanos.

```
CREATE OR REPLACE TRIGGER trg_validar_stock_detalle
BEFORE INSERT OR UPDATE ON detalle_ventas
FOR EACH ROW
DECLARE
    v_stock_actual NUMBER;
    v_nombre_prod  VARCHAR2(100);
BEGIN
    -- Bloqueo y obtención del stock actual del producto para control de concurrencia
    SELECT stock, nombre_producto
    INTO v_stock_actual, v_nombre_prod
    FROM productos
    WHERE id_producto = :NEW.id_producto
    FOR UPDATE; -- Garantiza aislamiento estricto

    -- Validación de la Regla de Negocio RN-02
    IF v_stock_actual < :NEW.cantidad THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Error de Inventario: Stock insuficiente para el producto [' || v_nombre_prod ||
            ']. Disponible: ' || v_stock_actual || ', Solicitado: ' || :NEW.cantidad);
    ELSE
        -- Descuento automático de stock integrado en el servidor
        UPDATE productos
        SET stock = stock - :NEW.cantidad
        WHERE id_producto = :NEW.id_producto;
    END IF;
END;
/
```

10. EXPLICACIÓN DE LOS PROCEDURES (PROCEDIMIENTOS ALMACENADOS)

Los procedimientos almacenados centralizan la lógica de procesamiento en el servidor de base de datos. Para este proyecto se desarrollaron dos procedimientos transaccionales complejos:

10.1. Procedure: registrar_venta_completa

Este objeto PL/SQL encapsula el procesamiento completo de una transacción de facturación comercial. Tradicionalmente, una venta requiere interactuar por separado con la tabla de cabeceras, insertar múltiples registros en los detalles y actualizar costos acumulados. Delegar esto a código externo genera riesgos de pérdida de conexión y corrupción de datos. El procedimiento recibe el código identificador del cliente y mediante lógica integrada ejecuta secuencialmente las operaciones. Al procesar las líneas de venta, se activa el trigger de validación de stock de manera transparente. Si ocurre alguna anomalía de inventario o violación de constraints, el bloque **EXCEPTION** atrapa el error e invoca un **ROLLBACK** integral, garantizando que no existan facturas parciales o registros huérfanos.

10.2. Procedure: registrar_reparacion

Automatiza los flujos operativos en el laboratorio técnico. Se encarga de verificar preventivamente la existencia del cliente en la entidad base. Al pasar los filtros, inserta la orden de asistencia técnica en la tabla **reparaciones** estableciendo el estado inicial reglamentario ('RECIBIDO'). Acto seguido, con el propósito de mantener la gobernanza corporativa de datos, realiza una inserción secundaria en la tabla **log_cambios** dejando constancia de la apertura del ticket, el operador a cargo y la marca temporal del hardware ingresado.

```

CREATE OR REPLACE PROCEDURE registrar_reparacion (
    p_id_cliente      IN NUMBER,
    p_dispositivo     IN VARCHAR2,
    p_descripcion     IN VARCHAR2,
    p_costo_estimado  IN NUMBER
) AS
    v_existe_cliente NUMBER;
    e_cliente_invalido EXCEPTION;
BEGIN
    -- Validación previa de integridad
    SELECT COUNT(1) INTO v_existe_cliente FROM clientes WHERE id_cliente = p_id_cliente;

    IF v_existe_cliente = 0 THEN
        RAISE e_cliente_invalido;
    END IF;

    -- Inserción controlada de la orden técnica
    INSERT INTO reparaciones (id_reparacion, id_cliente, dispositivo, descripcion_fallo, estado,
costo_estimado, fecha_ingreso)
    VALUES (seq_reparaciones.NEXTVAL, p_id_cliente, p_dispositivo, p_descripcion, 'RECIBIDO',
p_costo_estimado, SYSDATE);

    -- Registro de auditoría automática en el Log de Cambios
    INSERT INTO log_cambios (id_log, tabla_afectada, operacion, usuario, fecha, descripcion)
    VALUES (seq_log.NEXTVAL, 'REPARACIONES', 'INSERT', USER, SYSDATE,
'Ingreso de dispositivo técnico para reparación: ' || p_dispositivo || ' asignado al
cliente ID: ' || p_id_cliente);

    COMMIT; -- Persistencia duradera de los datos

EXCEPTION
    WHEN e_cliente_invalido THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20002, 'Violación de Integridad: El cliente ID [' ||
p_id_cliente || '] no está registrado en el sistema.');
```

11. EXPLICACIÓN DE ACID Y CONCURRENCIA AVANZADA

Para asegurar que DragonByte Mobile Store opere con estándares de fiabilidad industrial, la base de datos se rige estrictamente bajo el paradigma de las propiedades **ACID**, administradas mediante programación transaccional en Oracle:

- **Atomicidad (Atomicity):** Principio de "Todo o Nada". Si una venta incluye la salida de cinco periféricos diferentes y el último de ellos falla por stock, el sistema completo se retrotrae mediante **ROLLBACK**. No se guarda una venta "a medias".

- **Consistencia (Consistency):** Una transacción solo puede transformar la base de datos de un estado válido a otro estado válido. Las reglas se imponen mediante tipos de datos exactos, llaves foráneas y constraints de verificación (**CHECK**) imposibles de saltar por la capa de aplicación.
- **Aislamiento (Isolation):** Determina cómo las operaciones concurrentes son visibles para otros usuarios. Oracle utiliza mecanismos de aislamiento que previenen lecturas sucias y garantizan que los efectos de modificaciones concurrentes permanezcan invisibles hasta que se procese un **COMMIT** explícito.
- **Durabilidad (Durability):** Una vez que el sistema confirma una transacción al cliente, los datos quedan consolidados de forma permanente en almacenamiento no volátil (Redo Logs y Datafiles del motor), siendo inmunes a apagones eléctricos o fallos críticos del sistema operativo del servidor.

11.1. Gestión de Bloqueos y Robustez en Concurrency

Cuando cientos de solicitudes de compra saturan el servidor en tiempo real, el uso por defecto de bloqueos puede ralentizar las aplicaciones o generar interbloqueos destructivos. Para solucionar esto, el sistema emplea cláusulas avanzadas de adquisición de recursos:

1. Sentencia *FOR UPDATE NOWAIT*: Al ejecutar una consulta de selección de productos, esta instrucción instruye al motor Oracle a bloquear de forma exclusiva e inmediata las filas consultadas. Si otra sesión transaccional mantiene retenida alguna de esas filas, el motor no retiene la solicitud en cola de espera; por el contrario, aborta la operación instantáneamente arrojando el error de sistema **ORA-00054 (Resource Busy)**. Esto permite liberar los hilos de ejecución de la aplicación backend de inmediato sin congelar la interfaz de usuario.

2. Sentencia *FOR UPDATE WAIT 5*: Representa un enfoque intermedio flexible. Otorga al motor una ventana temporal estricta de tolerancia equivalente a 5 segundos de reloj para intentar capturar el bloqueo exclusivo sobre el registro de inventario. Si la transacción bloqueante previa se libera mediante commit o rollback dentro de ese lapso, la transacción actual continúa limpiamente; si el temporizador expira sin liberación, levanta la excepción de recurso ocupado. Esto mitiga de forma radical los cuellos de botella pesados.

3. Mecanismo de Compilación *PRAGMA EXCEPTION_INIT*: En PL/SQL, los errores nativos del motor se manifiestan mediante códigos numéricos crudos. Para proveer un desarrollo modular y limpio, esta directiva del compilador asocia un código de error numérico interno de Oracle (como el -54 para recursos bloqueados de forma concurrente) a un identificador de excepción semántico definido por el ingeniero. Esto permite capturar las colisiones concurrentes de inventario de forma estructurada en bloques de control ordinarios:

```

DECLARE
    -- Asociación de error numérico nativo a identificador semántico de programación
    e_recurso_ocupado EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_recurso_ocupado, -54);

    v_precio NUMBER;
BEGIN
    -- Intento inmediato de reserva exclusiva de stock para un celular de alta gama
    SELECT precio_unitario INTO v_precio
    FROM productos
    WHERE id_producto = 101
    FOR UPDATE NOWAIT; -- Intenta adquirir el bloqueo sin tolerancia a esperas

EXCEPTION
    WHEN e_recurso_ocupado THEN
        -- Control sofisticado de la concurrencia: la base de datos evita el congelamiento
        RAISE_APPLICATION_ERROR(-20054, 'El terminal o accesorio se encuentra bajo procesamiento
de pago por otra terminal de venta. Intente en unos instantes.');
```

END;

/

12. EXPLICACIÓN DE LAS VISTAS (CAPA LÓGICA DE ABSTRACCIÓN)

Las vistas representan abstracciones de datos lógicas almacenadas en el servidor que no duplican el almacenamiento físico, sino que actúan como consultas pre-compiladas de alto rendimiento. En este proyecto cumplen dos propósitos neurálgicos: seguridad perimetral y analítica gerencial.

12.1. Vista de Seguridad y Ocultamiento de Datos Sensibles: `v_catalogo_publico`

Desde la perspectiva de la seguridad informática, exponer de manera directa las tablas maestras de inventario a las aplicaciones conectadas a internet constituye una vulnerabilidad crítica. La vista `v_catalogo_publico` restringe el acceso al núcleo de la tabla `productos`, exponiendo exclusivamente el identificador, nombre comercial, descripción legible, precio de venta al público y la bandera lógica de disponibilidad. Campos confidenciales de auditoría interna, costos de importación de fábrica y stock de seguridad crítico permanecen completamente enmascarados, imposibilitando fugas de información estratégica hacia competidores u operadores no autorizados.

12.2. Vista de Reportes Empresariales: `v_reporte_ventas_detallado`

La toma de decisiones estratégicas por parte de la gerencia de DragonByte Mobile Store requiere consolidar métricas financieras complejas en tiempo real. Escribir sentencias con múltiples uniones cruzadas (`JOIN`) de forma repetitiva drena los recursos de procesamiento. Esta vista consolida las tablas transaccionales `ventas`, `detalle_ventas`, `clientes` y `productos` en una única estructura analítica denormalizada. Permite extraer reportes directos sobre el volumen total facturado, fechas de mayor demanda y comportamiento de consumo por usuario con una latencia óptima.

```
-- Estructuración de la Vista Analítica Gerencial
CREATE OR REPLACE VIEW v_reporte_ventas_detallado AS
SELECT
    v.id_venta,
    v.fecha_venta,
    c.id_cliente,
    (c.nombre || ' ' || c.apellido) AS cliente_completo,
    p.nombre_producto AS articulo_adquirido,
    p.categoria AS tipo_articulo,
    dv.cantidad AS unidades_compradas,
    dv.precio_unitario_aplicado AS precio_facturado,
    (dv.cantidad * dv.precio_unitario_aplicado) AS subtotal_linea,
    v.total_venta AS total_factura_global
FROM ventas v
JOIN clientes c ON v.id_cliente = c.id_cliente
JOIN detalle_ventas dv ON v.id_venta = dv.id_venta
JOIN productos p ON dv.id_producto = p.id_producto;
```


13. MATRIZ DE REQUERIMIENTOS DEL NEGOCIO VS OBJETOS SQL

La siguiente matriz técnica de trazabilidad conecta los requerimientos operativos de DragonByte Mobile Store con los componentes de software desarrollados e implementados en el servidor Oracle:

ID	NECESIDAD OPERATIVA / REQUERIMIENTO	SOLUCIÓN TÉCNICA IMPLEMENTADA	OBJETO SQL / PL-SQL RESPONSABLE
REQ-01	Garantizar que no se listen o vendan celulares con precios erróneos o en cero.	Restricciones de dominio restrictivas a nivel de columna (DDL).	Restricción CHECK (<i>precio_unitario > 0</i>) en tabla PRODUCTOS.
REQ-02	Evitar de forma automatizada la venta en negativo de repuestos y smartphones.	Evaluación en tiempo de ejecución interceptando la transacción antes de confirmarse.	Trigger trg_validar_stock_detalle a nivel de fila.
REQ-03	Procesar la compra de múltiples productos en un solo bloque indivisible y seguro.	Encapsulamiento transaccional con manejo avanzado de excepciones y rollback automático.	Procedure registrar_venta_completa .
REQ-04	Rastrear los cambios del taller y registrar ingresos de dispositivos de forma estandarizada.	Inserción procedimental acoplada a inserciones secundarias de auditoría en el sistema.	Procedure registrar_reparacion conectado a tabla LOG_CAMBIOS.
REQ-05	Proteger datos financieros e industriales contra accesos ilícitos del personal operativo.	Abstracción de consultas mediante enmascaramiento selectivo de atributos críticos.	Vista v_catalogo_publico .
REQ-06	Monitorear los indicadores de ventas mensuales y el rendimiento financiero por categorías.	Consolidación analítica indexada y pre-compilada de las transacciones comerciales.	Vista v_reporte_ventas_detallado .
REQ-07	Evitar el congelamiento del sistema web de ventas cuando hay alta saturación transaccional.	Políticas de control de concurrencia basadas en excepciones de expiración de tiempo.	Sentencias de bloqueo explícito FOR UPDATE NOWAIT / WAIT 5 .

14. CONCLUSIONES

1. **Gobernanza de Reglas en el Servidor:** Delegar la lógica transaccional y el control de inventarios a objetos nativos Oracle (Triggers y Procedures) garantiza una arquitectura monolítica de datos altamente resistente a fallos. Esto elimina la dependencia absoluta del comportamiento correcto de las aplicaciones de interfaz externa o backend.
2. **Erradicación de Inconsistencias Físicas:** Mediante el desarrollo del trigger `trg_validar_stock_detalle`, la tienda cuenta con un mecanismo infalible que anula la posibilidad de ventas simuladas o descuadres lógicos de inventario, protegiendo los activos fijos de la organización de forma automatizada.
3. **Garantía de Robustez Industrial:** El control estricto de transacciones mediante bloques PL/SQL estructurados con políticas ACID asegura la inmunidad de la base de datos ante interrupciones físicas de energía o de red, garantizando que toda operación comercial se procese de manera atómica ("todo o nada").
4. **Eficiencia en Concurrencia Saturada:** La implementación estratégica de mecanismos de bloqueo selectivo como `FOR UPDATE NOWAIT` y la manipulación de excepciones mediante `PRAGMA EXCEPTION_INIT` elimina los cuellos de botella críticos del sistema de ventas en horas pico, optimizando los tiempos de procesamiento y respuesta global.
5. **Seguridad por Aislamiento:** La arquitectura implementada demuestra que la seguridad en ingeniería de software debe nacer desde el modelado de datos. El uso de vistas como filtros de enmascaramiento blindo la persistencia contra filtraciones o espionaje corporativo involuntario del personal operativo de la tienda.

15. BUENAS PRÁCTICAS DE INGENIERÍA DE DATOS IMPLEMENTADAS

El código y la estructura física del proyecto se adhirieron de manera rigurosa a estándares internacionales de desarrollo en bases de datos empresariales:

- **Nomenclatura Estandarizada Estricta:** Uso de prefijos unívocos para la identificación de objetos del sistema (e.g., `trg_` para disparadores, `v_` para vistas, `seq_` para secuencias numéricas y variables locales con prefijo `v_` o parámetros con `p_`). Esto agiliza las auditorías de código y el mantenimiento a largo plazo.
- **Modularidad Funcional Avanzada:** Separación total de responsabilidades. Las operaciones comerciales no se ejecutan mediante bloques de código masivos y planos, sino a través de subprogramas parametrizados que aíslan la lógica operativa.
- **Gestión Centralizada y Sofisticada de Excepciones:** Prohibición absoluta del uso de bloques vacíos o capturas genéricas ineficientes. Cada procedimiento implementa manejadores específicos con llamadas a funciones nativas como `SQLERRM` y códigos personalizados, facilitando la trazabilidad de errores técnicos en producción.

- **Control Transaccional Limpio:** Las instrucciones de confirmación definitiva (*COMMIT*) o reversión completa (*ROLLBACK*) se ubican estrictamente al cierre de las operaciones lógicas lícitas o al inicio de los bloques de error, impidiendo que transacciones abiertas degraden el rendimiento de las tablas por bloqueos huérfanos.

16. RECOMENDACIONES FUTURAS Y EVOLUCIÓN DEL SISTEMA

Para expandir las capacidades operativas e informáticas de DragonByte Mobile Store hacia un ecosistema digital moderno a escala nacional, se proponen las siguientes líneas de evolución tecnológica:

1. **Desacoplamiento mediante API REST (Oracle ORDS):** Implementar *Oracle REST Data Services* para exponer los procedimientos almacenados (*registrar_venta_completa* y *registrar_reparacion*) como endpoints HTTP seguros en formato JSON, permitiendo la integración nativa y directa de aplicaciones web front-end y aplicaciones móviles multiplataforma sin dependencias de drivers pesados.
2. **Migración a un Ecosistema Web Responsivo y Omnicanal:** Desarrollar una interfaz de usuario moderna utilizando frameworks de vanguardia (como React o Next.js) que consuma la capa analítica de la base de datos, agilizando los tiempos de despacho y permitiendo el autoservicio de clientes para cotizaciones en línea.
3. **Módulo Avanzado de Autenticación y Auditoría Fina (Fine-Grained Auditing):** Integrar mecanismos de seguridad avanzados de Oracle para registrar de forma pormenorizada no solo los cambios de datos físicos, sino también los intentos de lectura (*SELECT*) sobre columnas críticas, asegurando trazabilidad forense total ante incidentes de ciberseguridad.
4. **Dashboard Analítico y Reportes Gráficos Interactivos:** Conectar la vista gerencial *v_reporte_ventas_detallado* a herramientas de Inteligencia de Negocios (BI) como Oracle APEX o Power BI, facilitando la visualización gráfica interactiva de los indicadores clave de rendimiento (KPIs), proyecciones de demanda y alertas de stock mínimo crítico.